Table of Contents

torch.utils.data

At the heart of PyTorch data loading utility is the `torch.utils.data.DataLoader` class. It represents a Python iterable over a dataset, with support for

- map-style and iterable-style datasets,
- customizing data loading order,
- automatic batching,
- single- and multi-process data loading,
- automatic memory pinning.

These options are configured by the constructor arguments of a `DataLoader`, which has signature:

```
DataLoader(dataset, batch_size=1, shuffle=False, sampler=None,
           batch_sampler=None, num_workers=0, collate_fn=None,
           pin_memory=False, drop_last=False, timeout=0,
           worker_init_fn=None, *, prefetch_factor=2,
           persistent_workers=False)
```

The sections below describe in details the effects and usages of these options.

Dataset Types

The most important argument of `DataLoader` constructor is `dataset`, which indicates a dataset object to load data from. PyTorch supports two different types of datasets:

- map-style datasets,
- iterable-style datasets.

Map-style datasets

A map-style dataset is one that implements the `__getitem__()` and `__len__()` protocols, and represents a map from (possibly non-integral) indices/keys to data samples.

For example, such a dataset, when accessed with `dataset[idx]`, could read the `idx`-th image and its corresponding label from a folder on the disk.

See `Dataset` for more details.

Iterable-style datasets

An iterable-style dataset is an instance of a subclass of `IterableDataset` that implements the `__iter__()` protocol, and represents an iterable over data samples. This type of datasets is particularly suitable for cases where **random** reads are expensive or even improbable, and where the batch size depends on the fetched data.

For example, such a dataset, when called `iter(dataset)`, could return a stream of data reading from a database, a remote server, or even logs generated in real time.

See `IterableDataset` for more details.

• NOTE

When using a `IterableDataset` with multi-process data loading. The same dataset object is replicated on each worker process, and thus the replicas must be configured differently to avoid duplicated data. See `IterableDataset` documentations for how to achieve this.

Data Loading Order and `Sampler`

For iterable-style datasets, data loading order is entirely controlled by the user-defined iterable. This allows easier implementations of chunk-reading and dynamic batch size (e.g., by yielding a batched sample at each time).

The rest of this section concerns the case with map-style datasets. `torch.utils.data.Sampler` classes are used to specify the sequence of indices/keys used in data loading. They represent iterable objects over the indices to datasets. E.g., in the common case with stochastic gradient decent (SGD), a `Sampler` could randomly permute a list of indices and yield each one at a time, or yield a small number of them for mini-batch SGD.

A sequential or shuffled sampler will be automatically constructed based on the `shuffle` argument to a `DataLoader`. Alternatively, users may use the `sampler` argument to specify a custom `Sampler` object that at each time yields the next index/key to fetch.

A custom `Sampler` that yields a list of batch indices at a time can be passed as the `batch_sampler` argument. Automatic batching can also be enabled via `batch_size` and `drop_last` arguments. See the next section for more details on this.

• NOTE

Neither `sampler` nor `batch_sampler` is compatible with iterable-style datasets, since such datasets have no notion of a key or an index.

Loading Batched and Non-Batched Data

`DataLoader` supports automatically collating individual fetched data samples into batches via arguments `batch_size`, `drop_last`, `batch_sampler`, and `collate_fn` (which has a default function).

Automatic batching (default)

This is the most common case, and corresponds to fetching a minibatch of data and collating them into batched samples, i.e., containing Tensors with one dimension being the batch dimension (usually the first).

When `batch_size` (default `1`) is not `None`, the data loader yields batched samples instead of individual samples. `batch_size` and `drop_last` arguments are used to specify how the data loader obtains batches of dataset keys. For map-style datasets, users can alternatively specify `batch_sampler`, which yields a list of keys at a time.

• NOTE

The `batch_size` and `drop_last` arguments essentially are used to construct a `batch_sampler` from `sampler`. For map-style datasets, the `sampler` is either provided by user or constructed based on the `shuffle` argument. For iterable-style datasets, the `sampler` is a dummy infinite one. See this section on more details on samplers.

• NOTE

When fetching from iterable-style datasets with multi-processing, the `drop_last` argument drops the last non-full batch of each worker's dataset replica.

After fetching a list of samples using the indices from sampler, the function passed as the `collate_fn` argument is used to collate lists of samples into batches.

In this case, loading from a map-style dataset is roughly equivalent with:

```
for indices in batch_sampler:
    yield collate_fn([dataset[i] for i in indices])
```

and loading from an iterable-style dataset is roughly equivalent with:

```
dataset_iter = iter(dataset)
for indices in batch_sampler:
    yield collate_fn([next(dataset_iter) for _ in indices])
```

A custom `collate_fn` can be used to customize collation, e.g., padding sequential data to max length of a batch. See this section on more about `collate_fn`.

Disable automatic batching

In certain cases, users may want to handle batching manually in dataset code, or simply load individual samples. For example, it could be cheaper to directly load batched data (e.g., bulk reads from a database or reading continuous chunks of memory), or the batch size is data dependent, or the program is designed to work on individual samples. Under these scenarios, it's likely better to not use automatic batching (where `collate_fn` is used to collate the samples), but let the data loader directly return each member of the `dataset` object.

When both `batch_size` and `batch_sampler` are `None` (default value for `batch_sampler` is already `None`), automatic batching is disabled. Each sample obtained from the `dataset` is processed with the function passed as the `collate_fn` argument.

When automatic batching is disabled, the default `collate_fn` simply converts NumPy arrays into PyTorch Tensors, and keeps everything else untouched.

In this case, loading from a map-style dataset is roughly equivalent with:

```
for index in sampler:
    yield collate_fn(dataset[index])
```

and loading from an iterable-style dataset is roughly equivalent with:

```
for data in iter(dataset):
    yield collate_fn(data)
```

See this section on more about `collate_fn`.

Working with `collate_fn`

The use of `collate_fn` is slightly different when automatic batching is enabled or disabled.

When automatic batching is disabled, `collate_fn` is called with each individual data sample, and the output is yielded from the data loader iterator. In this case, the default `collate_fn` simply converts NumPy arrays in PyTorch tensors.

When automatic batching is enabled, `collate_fn` is called with a list of data samples at each time. It is expected to collate the input samples into a batch for yielding from the data loader iterator. The rest of this section describes the behavior of the default `collate_fn` (`default_collate()`).

For instance, if each data sample consists of a 3-channel image and an integral class label, i.e., each element of the dataset returns a tuple `(image, class_index)`, the default `collate_fn` collates a list of such tuples into a single tuple of a batched image tensor and a batched class label Tensor. In particular, the default `collate_fn` has the following properties:

- It always prepends a new dimension as the batch dimension.
- It automatically converts NumPy arrays and Python numerical values into PyTorch Tensors.

- It preserves the data structure, e.g., if each sample is a dictionary, it outputs a dictionary with the same set of keys but batched Tensors as values (or lists if the values can not be converted into Tensors). Same for `list` s, `tuple` s, `namedtuple` s, etc.

Users may use customized `collate_fn` to achieve custom batching, e.g., collating along a dimension other than the first, padding sequences of various lengths, or adding support for custom data types.

If you run into a situation where the outputs of `DataLoader` have dimensions or type that is different from your expectation, you may want to check your `collate_fn` .

Single- and Multi-process Data Loading

A `DataLoader` uses single-process data loading by default.

Within a Python process, the Global Interpreter Lock (GIL) prevents true fully parallelizing Python code across threads. To avoid blocking computation code with data loading, PyTorch provides an easy switch to perform multi-process data loading by simply setting the argument `num_workers` to a positive integer.

Single-process data loading (default)

In this mode, data fetching is done in the same process a `DataLoader` is initialized. Therefore, data loading may block computing. However, this mode may be preferred when resource(s) used for sharing data among processes (e.g., shared memory, file descriptors) is limited, or when the entire dataset is small and can be loaded entirely in memory. Additionally, single-process loading often shows more readable error traces and thus is useful for debugging.

Multi-process data loading

Setting the argument `num_workers` as a positive integer will turn on multi-process data loading with the specified number of loader worker processes.

• WARNING

After several iterations, the loader worker processes will consume the same amount of CPU memory as the parent process for all Python objects in the parent process which are accessed from the worker processes. This can be problematic if the Dataset contains a lot of data (e.g., you are loading a very large list of filenames at Dataset construction time) and/or you are using a lot of workers (overall memory usage is `number of workers * size of parent process`). The simplest workaround is to replace Python objects with non-refcounted representations such as Pandas, Numpy or PyArrow objects. Check out issue #13246 for more details on why this occurs and example code for how to workaround these problems.

In this mode, each time an iterator of a `DataLoader` is created (e.g., when you call `enumerate(dataloader)`), `num_workers` worker processes are created. At this point, the `dataset` , `collate_fn` , and `worker_init_fn` are passed to each worker, where they are used to initialize, and fetch data. This means that dataset access together with its internal IO, transforms (including `collate_fn`) runs in the worker process.

`torch.utils.data.get_worker_info()` returns various useful information in a worker process (including the worker id, dataset replica, initial seed, etc.), and returns `None` in main process. Users may use this function in dataset code and/or `worker_init_fn` to individually configure each dataset replica, and to determine whether the code is running in a worker process. For example, this can be particularly helpful in sharding the dataset.

For map-style datasets, the main process generates the indices using `sampler` and sends them to the workers. So any shuffle randomization is done in the main process which guides loading by assigning indices to load.

For iterable-style datasets, since each worker process gets a replica of the `dataset` object, naive multi-process loading will often result in duplicated data. Using `torch.utils.data.get_worker_info()` and/or `worker_init_fn` , users may configure each replica independently. (See `IterableDataset` documentations for how to achieve this.) For similar reasons, in multi-process loading, the `drop_last` argument drops the last non-full batch of each worker's iterable-style dataset replica.

Workers are shut down once the end of the iteration is reached, or when the iterator becomes garbage collected.

• WARNING

It is generally not recommended to return CUDA tensors in multi-process loading because of many subtleties in using CUDA and sharing CUDA tensors in multiprocessing (see CUDA in multiprocessing). Instead, we recommend using automatic memory pinning (i.e., setting `pin_memory=True`), which enables fast data transfer to CUDA-enabled GPUs.

Platform-specific behaviors

Since workers rely on Python `multiprocessing` , worker launch behavior is different on Windows compared to Unix.

- On Unix, `fork()` is the default `multiprocessing` start method. Using `fork()` , child workers typically can access the `dataset` and Python argument functions directly through the cloned address space.
- On Windows or MacOS, `spawn()` is the default `multiprocessing` start method. Using `spawn()` , another interpreter is launched which runs your main script, followed by the internal worker function that receives the `dataset` , `collate_fn` and other arguments through `pickle` serialization.

This separate serialization means that you should take two steps to ensure you are compatible with Windows while using multi-process data loading:

- Wrap most of you main script's code within `if __name__ == '__main__':` block, to make sure it doesn't run again (most likely generating error) when each worker process is launched. You can place your dataset and `DataLoader` instance creation logic here, as it doesn't need to be re-executed in workers.
- Make sure that any custom `collate_fn` , `worker_init_fn` or `dataset` code is declared as top level definitions, outside of the `__main__` check. This ensures that they are available in worker processes. (this is needed since functions are pickled as references only, not `bytecode` .)

Randomness in multi-process data loading

By default, each worker will have its PyTorch seed set to `base_seed + worker_id` , where `base_seed` is a long generated by main process using its RNG (thereby, consuming a RNG state mandatorily) or a specified `generator` . However, seeds for other libraries may be duplicated upon initializing workers, causing each worker to return identical random numbers. (See this section in FAQ.).

In `worker_init_fn` , you may access the PyTorch seed set for each worker with either `torch.utils.data.get_worker_info().seed` or `torch.initial_seed()` , and use it to seed other libraries before data loading.

Memory Pinning

Host to GPU copies are much faster when they originate from pinned (page-locked) memory. See Use pinned memory buffers for more details on when and how to use pinned memory generally.

For data loading, passing `pin_memory=True` to a `DataLoader` will automatically put the fetched data Tensors in pinned memory, and thus enables faster data transfer to CUDA-enabled GPUs.

The default memory pinning logic only recognizes Tensors and maps and iterables containing Tensors. By default, if the pinning logic sees a batch that is a custom type (which will occur if you have a `collate_fn` that returns a custom batch type), or if each element of your batch is a custom type, the pinning logic will not recognize them, and it will return that batch (or those elements) without pinning the memory. To enable memory pinning for custom batch or data type(s), define a `pin_memory()` method on your custom type(s).

See the example below.

Example:

```
class SimpleCustomBatch:
    def __init__(self, data):
        transposed_data = list(zip(*data))
        self.inp = torch.stack(transposed_data[0], 0)
        self.tgt = torch.stack(transposed_data[1], 0)

    # custom memory pinning method on custom type
    def pin_memory(self):
        self.inp = self.inp.pin_memory()
        self.tgt = self.tgt.pin_memory()
        return self

def collate_wrapper(batch):
    return SimpleCustomBatch(batch)

inps = torch.arange(10 * 5, dtype=torch.float32).view(10, 5)
tgts = torch.arange(10 * 5, dtype=torch.float32).view(10, 5)
dataset = TensorDataset(inps, tgts)

loader = DataLoader(dataset, batch_size=2, collate_fn=collate_wrapper,
                    pin_memory=True)

for batch_ndx, sample in enumerate(loader):
    print(sample.inp.is_pinned())
    print(sample.tgt.is_pinned())
```

CLASS torch.utils.data.DataLoader(*dataset*, *batch_size=1*, *shuffle=None*, *sampler=None*, *batch_sampler=None*, *num_workers=0*, *collate_fn=None*, *pin_memory=False*, *drop_last=False*, *timeout=0*, *worker_init_fn=None*, *multiprocessing_context=None*, *generator=None*, ***, *prefetch_factor=None*, *persistent_workers=False*, *pin_memory_device=''*, *in_order=True*) [SOURCE]

Data loader combines a dataset and a sampler, and provides an iterable over the given dataset.

The `DataLoader` supports both map-style and iterable-style datasets with single- or multi-process loading, customizing loading order and optional automatic batching (collation) and memory pinning.

See `torch.utils.data` documentation page for more details.

Parameters

- **dataset** (*Dataset*) – dataset from which to load the data.
- **batch_size** (*int, optional*) – how many samples per batch to load (default: `1`).
- **shuffle** (*bool, optional*) – set to `True` to have the data reshuffled at every epoch (default: `False`).
- **sampler** (*Sampler or Iterable, optional*) – defines the strategy to draw samples from the dataset. Can be any `Iterable` with `__len__` implemented. If specified, `shuffle` must not be specified.
- **batch_sampler** (*Sampler or Iterable, optional*) – like `sampler`, but returns a batch of indices at a time. Mutually exclusive with `batch_size`, `shuffle`, `sampler`, and `drop_last`.
- **num_workers** (*int, optional*) – how many subprocesses to use for data loading. `0` means that the data will be loaded in the main process. (default: `0`)
- **collate_fn** (*Callable, optional*) – merges a list of samples to form a mini-batch of Tensor(s). Used when using batched loading from a map-style dataset.
- **pin_memory** (*bool, optional*) – If `True`, the data loader will copy Tensors into device/CUDA pinned memory before returning them. If your data elements are a custom type, or your `collate_fn` returns a batch that is a custom type, see the example below.
- **drop_last** (*bool, optional*) – set to `True` to drop the last incomplete batch, if the dataset size is not divisible by the batch size. If `False` and the size of dataset is not divisible by the batch size, then the last batch will be smaller. (default: `False`)
- **timeout** (*numeric, optional*) – if positive, the timeout value for collecting a batch from workers. Should always be non-negative. (default: `0`)
- **worker_init_fn** (*Callable, optional*) – If not `None`, this will be called on each worker subprocess with the worker id (an int in `[0, num_workers - 1]`) as input, after seeding and before data loading. (default: `None`)
- **multiprocessing_context** (*str or multiprocessing.context.BaseContext, optional*) – If `None`, the default multiprocessing context of your operating system will be used. (default: `None`)
- **generator** (*torch.Generator, optional*) – If not `None`, this RNG will be used by RandomSampler to generate random indexes and multiprocessing to generate `base_seed` for workers. (default: `None`)
- **prefetch_factor** (*int, optional, keyword-only arg*) – Number of batches loaded in advance by each worker. `2` means there will be a total of 2 * num_workers batches prefetched across all workers. (default value depends on the set value for num_workers. If value of num_workers=0 default is `None`. Otherwise, if value of `num_workers > 0` default is `2`).
- **persistent_workers** (*bool, optional*) – If `True`, the data loader will not shut down the worker processes after a dataset has been consumed once. This allows to maintain the workers *Dataset* instances alive. (default: `False`)
- **pin_memory_device** (*str, optional*) – the device to `pin_memory` to if `pin_memory` is `True`.
- **in_order** (*bool, optional*) – If `False`, the data loader will not enforce that batches are returned in a first-in, first-out order. Only applies when `num_workers > 0`. (default: `True`)

• WARNING

If the `spawn` start method is used, `worker_init_fn` cannot be an unpicklable object, e.g., a lambda function. See Multiprocessing best practices on more details related to multiprocessing in PyTorch.

• WARNING

`len(dataloader)` heuristic is based on the length of the sampler used. When `dataset` is an `IterableDataset`, it instead returns an estimate based on `len(dataset) / batch_size`, with proper rounding depending on `drop_last`, regardless of multi-process loading configurations. This represents the best guess PyTorch can make because PyTorch trusts user `dataset` code in correctly handling multi-process loading to avoid duplicate data.

However, if sharding results in multiple workers having incomplete last batches, this estimate can still be inaccurate, because (1) an otherwise complete batch can be broken into multiple ones and (2) more than one batch worth of samples can be dropped when `drop_last` is set. Unfortunately, PyTorch can not detect such cases in general.

See Dataset Types for more details on these two types of datasets and how `IterableDataset` interacts with Multi-process data loading.

• WARNING

See Reproducibility, and My data loader workers return identical **random** numbers, and Randomness in multi-process data loading notes for **random** seed related questions.

• WARNING

Setting *in_order* to *False* can harm reproducibility and may lead to a skewed data distribution being fed to the trainer in cases with imbalanced data.

CLASS torch.utils.data.Dataset [SOURCE]

An abstract class representing a `Dataset`.

All datasets that represent a map from keys to data samples should subclass it. All subclasses should overwrite `__getitem__()`, supporting fetching a data sample for a given key. Subclasses could also optionally overwrite `__len__()`, which is expected to return the size of the dataset by many `Sampler` implementations and the default options of `DataLoader`. Subclasses could also optionally implement `__getitems__()`, for speedup batched samples loading. This method accepts list of indices of samples of batch and returns list of samples.

• NOTE

`DataLoader` by default constructs an index sampler that yields integral indices. To make it work with a map-style dataset with non-integral indices/keys, a custom sampler must be provided.

CLASS torch.utils.data.IterableDataset [SOURCE]

An iterable Dataset.

All datasets that represent an iterable of data samples should subclass it. Such form of datasets is particularly useful when data come from a stream.

All subclasses should overwrite `__iter__()`, which would return an iterator of samples in this dataset.

When a subclass is used with `DataLoader`, each item in the dataset will be yielded from the `DataLoader` iterator. When `num_workers > 0`, each worker process will have a different copy of the dataset object, so it is often desired to configure each copy independently to avoid having duplicate data returned from the workers. `get_worker_info()`, when called in a worker process, returns information about the worker. It can be used in either the dataset's `__iter__()` method or the `DataLoader` 's `worker_init_fn` option to modify each copy's behavior.

Example 1: splitting workload across all workers in `__iter__()`:

```
>>> class MyIterableDataset(torch.utils.data.IterableDataset):
...     def __init__(self, start, end):
...         super(MyIterableDataset).__init__()
...         assert end > start, "this example code only works with end >= start"
...         self.start = start
...         self.end = end
...
...     def __iter__(self):
...         worker_info = torch.utils.data.get_worker_info()
...         if worker_info is None:  # single-process data loading, return the full iterator
...             iter_start = self.start
...             iter_end = self.end
...         else:  # in a worker process
...             # split workload
...             per_worker = int(math.ceil((self.end - self.start) / float(worker_info.num_workers)))
...             worker_id = worker_info.id
...             iter_start = self.start + worker_id * per_worker
...             iter_end = min(iter_start + per_worker, self.end)
...         return iter(range(iter_start, iter_end))
...
>>> # should give same set of data as range(3, 7), i.e., [3, 4, 5, 6].
>>> ds = MyIterableDataset(start=3, end=7)

>>> # Single-process loading
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=0)))
[tensor([3]), tensor([4]), tensor([5]), tensor([6])]

>>> # Mult-process loading with two worker processes
>>> # Worker 0 fetched [3, 4].  Worker 1 fetched [5, 6].
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=2)))
[tensor([3]), tensor([5]), tensor([4]), tensor([6])]

>>> # With even more workers
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=12)))
[tensor([3]), tensor([5]), tensor([4]), tensor([6])]
```

Example 2: splitting workload across all workers using `worker_init_fn` :

```
>>> class MyIterableDataset(torch.utils.data.IterableDataset):
...     def __init__(self, start, end):
...         super(MyIterableDataset).__init__()
...         assert end > start, "this example code only works with end >= start"
...         self.start = start
...         self.end = end
...
...     def __iter__(self):
...         return iter(range(self.start, self.end))
...
>>> # should give same set of data as range(3, 7), i.e., [3, 4, 5, 6].
>>> ds = MyIterableDataset(start=3, end=7)

>>> # Single-process loading
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=0)))
[3, 4, 5, 6]
>>>
>>> # Directly doing multi-process loading yields duplicate data
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=2)))
[3, 3, 4, 4, 5, 5, 6, 6]

>>> # Define a `worker_init_fn` that configures each dataset copy differently
>>> def worker_init_fn(worker_id):
...     worker_info = torch.utils.data.get_worker_info()
...     dataset = worker_info.dataset  # the dataset copy in this worker process
...     overall_start = dataset.start
...     overall_end = dataset.end
...     # configure the dataset to only process the split workload
...     per_worker = int(math.ceil((overall_end - overall_start) / float(worker_info.num_workers)))
...     worker_id = worker_info.id
...     dataset.start = overall_start + worker_id * per_worker
...     dataset.end = min(dataset.start + per_worker, overall_end)
...

>>> # Mult-process loading with the custom `worker_init_fn`
>>> # Worker 0 fetched [3, 4].  Worker 1 fetched [5, 6].
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=2, worker_init_fn=worker_init_fn)))
[3, 5, 4, 6]

>>> # With even more workers
>>> print(list(torch.utils.data.DataLoader(ds, num_workers=12, worker_init_fn=worker_init_fn)))
[3, 4, 5, 6]
```

CLASS torch.utils.data.TensorDataset(**tensors*) [SOURCE]

Dataset wrapping tensors.

Each sample will be retrieved by indexing tensors along the first dimension.

Parameters

***tensors** (*Tensor*) – tensors that have the same size of the first dimension.

CLASS torch.utils.data.StackDataset(**args*, ***kwargs*) [SOURCE]

Dataset as a stacking of multiple datasets.

This class is useful to assemble different parts of complex input data, given as datasets.

Example

```
>>> images = ImageDataset()
>>> texts = TextDataset()
>>> tuple_stack = StackDataset(images, texts)
>>> tuple_stack[0] == (images[0], texts[0])
>>> dict_stack = StackDataset(image=images, text=texts)
>>> dict_stack[0] == {'image': images[0], 'text': texts[0]}
```

Parameters

- ***args** (*Dataset*) – Datasets for stacking returned as tuple.
- ****kwargs** (*Dataset*) – Datasets for stacking returned as dict.

CLASS torch.utils.data.ConcatDataset(*datasets*) [SOURCE]

Dataset as a concatenation of multiple datasets.

This class is useful to assemble different existing datasets.

Parameters

datasets (*sequence*) – List of datasets to be concatenated

CLASS torch.utils.data.ChainDataset(*datasets*) [SOURCE]

Dataset for chaining multiple `IterableDataset` s.

This class is useful to assemble different existing dataset streams. The chaining operation is done on-the-fly, so concatenating large-scale datasets with this class will be efficient.

Parameters

datasets (*iterable of IterableDataset*) – datasets to be chained together

CLASS torch.utils.data.Subset(*dataset*, *indices*) [SOURCE]

Subset of a dataset at specified indices.

Parameters

- **dataset** (*Dataset*) – The whole Dataset
- **indices** (*sequence*) – Indices in the whole set selected for subset

torch.utils.data._utils.collate.collate(*batch*, ***, *collate_fn_map=None*) [SOURCE]

General collate function that handles collection type of element within each batch.

The function also opens function registry to deal with specific element types. *default_collate_fn_map* provides default collate functions for tensors, numpy arrays, numbers and strings.

Parameters

- **batch** – a single batch to be collated
- **collate_fn_map** (*Optional*[*Dict*[*Union*[*Type*, *Tuple*[*Type*, ...]], *Callable*]]) – Optional dictionary mapping from element type to the corresponding collate function. If the element type isn't present in this dictionary, this function will go through each key of the dictionary in the insertion order to invoke the corresponding collate function if the element type is a subclass of the key.

Examples

```
>>> def collate_tensor_fn(batch, *, collate_fn_map):
...     # Extend this function to handle batch of tensors
...     return torch.stack(batch, 0)
>>> def custom_collate(batch):
...     collate_map = {torch.Tensor: collate_tensor_fn}
...     return collate(batch, collate_fn_map=collate_map)
>>> # Extend `default_collate` by in-place modifying `default_collate_fn_map`
>>> default_collate_fn_map.update({torch.Tensor: collate_tensor_fn})
```

• NOTE

Each collate function requires a positional argument for batch and a keyword argument for the dictionary of collate functions as *collate_fn_map*.

torch.utils.data.default_collate(*batch*) [SOURCE]

Take in a batch of data and put the elements within the batch into a tensor with an additional outer dimension - batch size.

The exact output type can be a `torch.Tensor`, a *Sequence* of `torch.Tensor`, a Collection of `torch.Tensor`, or left unchanged, depending on the input type. This is used as the default function for collation when *batch_size* or *batch_sampler* is defined in `DataLoader`.

Here is the general input type (based on the type of the element within the batch) to output type mapping:

- `torch.Tensor` -> `torch.Tensor` (with an added outer dimension batch size)
- NumPy Arrays -> `torch.Tensor`
- *float* -> `torch.Tensor`
- *int* -> `torch.Tensor`
- *str* -> *str* (unchanged)
- *bytes* -> *bytes* (unchanged)
- *Mapping*[*K*, *V_i*] -> *Mapping*[*K*, *default_collate*([*V_1*, *V_2*, ...])]
- *NamedTuple*[*V1_i*, *V2_i*, ...] -> *NamedTuple*[*default_collate*([*V1_1*, *V1_2*, ...]), *default_collate*([*V2_1*, *V2_2*, ...]), ...]
- *Sequence*[*V1_i*, *V2_i*, ...] -> *Sequence*[*default_collate*([*V1_1*, *V1_2*, ...]), *default_collate*([*V2_1*, *V2_2*, ...]), ...]

Parameters

batch – a single batch to be collated

Examples

```
>>> # Example with a batch of `int`s:
>>> default_collate([0, 1, 2, 3])
tensor([0, 1, 2, 3])
>>> # Example with a batch of `str`s:
>>> default_collate(['a', 'b', 'c'])
['a', 'b', 'c']
>>> # Example with `Map` inside the batch:
>>> default_collate([{'A': 0, 'B': 1}, {'A': 100, 'B': 100}])
{'A': tensor([  0, 100]), 'B': tensor([  1, 100])}
>>> # Example with `NamedTuple` inside the batch:
>>> Point = namedtuple('Point', ['x', 'y'])
>>> default_collate([Point(0, 0), Point(1, 1)])
Point(x=tensor([0, 1]), y=tensor([0, 1]))
>>> # Example with `Tuple` inside the batch:
>>> default_collate([(0, 1), (2, 3)])
[tensor([0, 2]), tensor([1, 3])]
>>> # Example with `List` inside the batch:
>>> default_collate([[0, 1], [2, 3]])
[tensor([0, 2]), tensor([1, 3])]
>>> # Two options to extend `default_collate` to handle specific type
>>> # Option 1: Write custom collate function and invoke `default_collate`
>>> def custom_collate(batch):
...     elem = batch[0]
...     if isinstance(elem, CustomType):  # Some custom condition
...         return ...
...     else:  # Fall back to `default_collate`
...         return default_collate(batch)
>>> # Option 2: In-place modify `default_collate_fn_map`
>>> def collate_customtype_fn(batch, *, collate_fn_map=None):
...     return ...
>>> default_collate_fn_map.update(CustomType, collate_customtype_fn)
>>> default_collate(batch)  # Handle `CustomType` automatically
```

torch.utils.data.default_convert(*data*) [SOURCE]

Convert each NumPy array element into a `torch.Tensor`.

If the input is a *Sequence*, *Collection*, or *Mapping*, it tries to convert each element inside to a `torch.Tensor`. If the input is not an NumPy array, it is left unchanged. This is used as the default function for collation when both *batch_sampler* and *batch_size* are NOT defined in `DataLoader`.

The general input type to output type mapping is similar to that of `default_collate()`. See the description there for more details.

Parameters

data – a single data point to be converted

Examples

```
>>> # Example with `int`
>>> default_convert(0)
0
>>> # Example with NumPy array
>>> default_convert(np.array([0, 1]))
tensor([0, 1])
>>> # Example with NamedTuple
>>> Point = namedtuple('Point', ['x', 'y'])
>>> default_convert(Point(0, 0))
Point(x=0, y=0)
>>> default_convert(Point(np.array(0), np.array(0)))
Point(x=tensor(0), y=tensor(0))
>>> # Example with List
>>> default_convert([np.array([0, 1]), np.array([2, 3])])
[tensor([0, 1]), tensor([2, 3])]
```

torch.utils.data.get_worker_info() [SOURCE]

Returns the information about the current `DataLoader` iterator worker process.

When called in a worker, this returns an object guaranteed to have the following attributes:

- `id`: the current worker id.
- `num_workers`: the total number of workers.
- `seed`: the **random** seed set for the current worker. This value is determined by main process RNG and the worker id. See `DataLoader`'s documentation for more details.
- `dataset`: the copy of the dataset object in **this** process. Note that this will be a different object in a different process than the one in the main process.

When called in the main process, this returns `None`.

• NOTE

When used in a `worker_init_fn` passed over to `DataLoader`, this method can be useful to set up each worker process differently, for instance, using `worker_id` to configure the `dataset` object to only read a specific fraction of a sharded dataset, or use `seed` to seed other libraries used in dataset code.

Return type

Optional[*WorkerInfo*]

torch.utils.data.random_split(*dataset*, *lengths*, *generator=<torch._C.Generator object>*) [SOURCE]

Randomly split a dataset into non-overlapping new datasets of given lengths.

If a list of fractions that sum up to 1 is given, the lengths will be computed automatically as floor(frac * len(dataset)) for each fraction provided.

After computing the lengths, if there are any remainders, 1 count will be distributed in round-robin fashion to the lengths until there are no remainders left.

Optionally fix the generator for reproducible results, e.g.:

Example

```
>>> generator1 = torch.Generator().manual_seed(42)
>>> generator2 = torch.Generator().manual_seed(42)
>>> random_split(range(10), [3, 7], generator=generator1)
>>> random_split(range(30), [0.3, 0.3, 0.4], generator=generator2)
```

Parameters

- **dataset** (*Dataset*) – Dataset to be split
- **lengths** (*sequence*) – lengths or fractions of splits to be produced
- **generator** (*Generator*) – Generator used for the random permutation.

Return type

List[*Subset*[*_T*]]

CLASS torch.utils.data.Sampler(*data_source=None*) [SOURCE]

Base class for all Samplers.

Every Sampler subclass has to provide an `__iter__()` method, providing a way to iterate over indices or lists of indices (batches) of dataset elements, and may provide a `__len__()` method that returns the length of the returned iterators.

Parameters

data_source (*Dataset*) – This argument is not used and will be removed in 2.2.0. You may still have custom implementation that utilizes it.

Example

```
>>> class AccedingSequenceLengthSampler(Sampler[int]):
>>>     def __init__(self, data: List[str]) -> None:
>>>         self.data = data
>>>
>>>     def __len__(self) -> int:
>>>         return len(self.data)
>>>
>>>     def __iter__(self) -> Iterator[int]:
>>>         sizes = torch.tensor([len(x) for x in self.data])
>>>         yield from torch.argsort(sizes).tolist()
>>>
>>> class AccedingSequenceLengthBatchSampler(Sampler[List[int]]):
>>>     def __init__(self, data: List[str], batch_size: int) -> None:
>>>         self.data = data
>>>         self.batch_size = batch_size
>>>
>>>     def __len__(self) -> int:
>>>         return (len(self.data) + self.batch_size - 1) // self.batch_size
>>>
>>>     def __iter__(self) -> Iterator[List[int]]:
>>>         sizes = torch.tensor([len(x) for x in self.data])
>>>         for batch in torch.chunk(torch.argsort(sizes), len(self)):
>>>             yield batch.tolist()
```

• NOTE

The `__len__()` method isn't strictly required by `DataLoader`, but is expected in any calculation involving the length of a `DataLoader`.

CLASS torch.utils.data.SequentialSampler(*data_source*) [SOURCE]

Samples elements sequentially, always in the same order.

Parameters

data_source (*Dataset*) – dataset to sample from

CLASS torch.utils.data.RandomSampler(*data_source, replacement=False, num_samples=None, generator=None*) [SOURCE]

Samples elements randomly. If without replacement, then sample from a shuffled dataset.

If with replacement, then user can specify `num_samples` to draw.

Parameters

- **data_source** (*Dataset*) – dataset to sample from
- **replacement** (*bool*) – samples are drawn on-demand with replacement if `True`, default=``False``
- **num_samples** (*int*) – number of samples to draw, default=`len(dataset)`.
- **generator** (*Generator*) – Generator used in sampling.

CLASS torch.utils.data.SubsetRandomSampler(*indices, generator=None*) [SOURCE]

Samples elements randomly from a given list of indices, without replacement.

Parameters

- **indices** (*sequence*) – a sequence of indices
- **generator** (*Generator*) – Generator used in sampling.

CLASS torch.utils.data.WeightedRandomSampler(*weights*, *num_samples*, *replacement=True*, *generator=None*) [SOURCE]

Samples elements from `[0,..,len(weights)-1]` with given probabilities (weights).

Parameters

- **weights** (*sequence*) – a sequence of weights, not necessary summing up to one
- **num_samples** (*int*) – number of samples to draw
- **replacement** (*bool*) – if `True` , samples are drawn with replacement. If not, they are drawn without replacement, which means that when a sample index is drawn for a row, it cannot be drawn again for that row.
- **generator** (*Generator*) – Generator used in sampling.

Example

```
>>> list(WeightedRandomSampler([0.1, 0.9, 0.4, 0.7, 3.0, 0.6], 5, replacement=True))
[4, 4, 1, 4, 5]
>>> list(WeightedRandomSampler([0.9, 0.4, 0.05, 0.2, 0.3, 0.1], 5, replacement=False))
[0, 1, 4, 3, 2]
```

CLASS torch.utils.data.BatchSampler(*sampler*, *batch_size*, *drop_last*) [SOURCE]

Wraps another sampler to yield a mini-batch of indices.

Parameters

- **sampler** (*Sampler or Iterable*) – Base sampler. Can be any iterable object
- **batch_size** (*int*) – Size of mini-batch.
- **drop_last** (*bool*) – If `True` , the sampler will drop the last batch if its size would be less than `batch_size`

Example

```
>>> list(BatchSampler(SequentialSampler(range(10)), batch_size=3, drop_last=False))
[[0, 1, 2], [3, 4, 5], [6, 7, 8], [9]]
>>> list(BatchSampler(SequentialSampler(range(10)), batch_size=3, drop_last=True))
[[0, 1, 2], [3, 4, 5], [6, 7, 8]]
```

CLASS torch.utils.data.distributed.DistributedSampler(*dataset*, *num_replicas=None*, *rank=None*, *shuffle=True*, *seed=0*, *drop_last=False*) [SOURCE]

Sampler that restricts data loading to a subset of the dataset.

It is especially useful in conjunction with `torch.nn.parallel.DistributedDataParallel` . In such a case, each process can pass a `DistributedSampler` instance as a `DataLoader` sampler, and load a subset of the original dataset that is exclusive to it.

• NOTE

Dataset is assumed to be of constant size and that any instance of it always returns the same elements in the same order.

Parameters

- **dataset** (*Dataset*) – Dataset used for sampling.
- **num_replicas** (*int, optional*) – Number of processes participating in distributed training. By default, `world_size` is retrieved from the current distributed group.
- **rank** (*int, optional*) – Rank of the current process within `num_replicas` . By default, `rank` is retrieved from the current distributed group.
- **shuffle** (*bool, optional*) – If `True` (default), sampler will shuffle the indices.
- **seed** (*int, optional*) – random seed used to shuffle the sampler if `shuffle=True` . This number should be identical across all processes in the distributed group. Default: `0` .
- **drop_last** (*bool, optional*) – if `True` , then the sampler will drop the tail of the data to make it evenly divisible across the number of replicas. If `False` , the sampler will add extra indices to make the data evenly divisible across the replicas. Default: `False` .

• WARNING

In distributed mode, calling the `set_epoch()` method at the beginning of each epoch **before** creating the `DataLoader` iterator is necessary to make shuffling work properly across multiple epochs. Otherwise, the same ordering will be always used.

Example:

```
>>> sampler = DistributedSampler(dataset) if is_distributed else None
>>> loader = DataLoader(dataset, shuffle=(sampler is None),
...                     sampler=sampler)
>>> for epoch in range(start_epoch, n_epochs):
...     if is_distributed:
...         sampler.set_epoch(epoch)
...     train(loader)
```

Previous Next